

ABOVE US

Game Agent Requirements & Direction Package

Version: 0.1

Status: Foundation Document

Primary Owner: Klaasvaakie

Primary Agent Role: Autonomous Game Designer, Developer, and Systems Builder

1. Core Concept

Above Us is a stylised isometric influence simulation game.

The player is an unseen entity observing a modern, exaggerated city-like world from above. The player does not begin as a visible character inside the world. Instead, they act as a presence that can influence people, events, emotions, decisions, relationships, and outcomes.

The game combines the godlike perspective and influence fantasy of a god game with the human tension, attraction, awkwardness, and relationship-driven outcomes of a social simulation.

The game must not be designed as a simple adult game, dating game, or card game. Its main purpose is not direct intimacy. Its main purpose is to let the player discover what kind of unseen force they become through the way they influence others.

Intimate, romantic, emotionally charged, or sensual moments may exist, but they must be earned through systems such as connection, consent, alignment, timing, context, trust, and character state.

Core idea:

The player shapes the world from above, and the world reveals who the player is becoming.

2. Genre Definition

The game is best described as:

Isometric Influence Simulation with Emergent Relationships

Secondary genre influences:

- God game

- Life simulation
- Relationship simulation
- Narrative systems game
- Strategy simulation
- Character evolution game
- Light adult-themed progression game, if platform rules allow

The game should feel strategic, personal, reactive, replayable, and expandable.

3. Target Tone

The first release uses a **stylised modern-life setting**.

The tone should be:

- Modern
- Relatable
- Slightly exaggerated
- Stylish
- Emotionally reactive
- Playful but not childish
- Seductive where appropriate, but not cheap
- Dramatic enough to create interesting situations
- Systemic enough to support replayability

The game should not feel like pure realism. It should feel like a slightly heightened version of modern social life where emotions, opportunities, attraction, fear, ambition, jealousy, and transformation are easier to see and influence.

4. Long-Term Product Strategy

The base game should launch with a modern setting because modern life is immediately understandable to players.

Later expansions can reuse the same core engine while changing the world theme, character archetypes, visuals, event data, and setting-specific rules.

Possible expansion worlds:

- Medieval
- Cyberpunk
- Fantasy with elves, dwarves, magical societies, courts, guilds, and clans
- Corporate dystopia

- Nightlife city
- Mythic dream world
- Supernatural modern world
- Post-apocalyptic settlement

The engine must be designed so that worlds are mostly data packs, not separate games.

Core rule:

The engine stays stable. Worlds change the data, art, rules, tone, and scenarios.

5. Primary Player Fantasy

The player fantasy is not simply “seduce characters.”

The real fantasy is:

I am above the world. I can observe, influence, shape, protect, corrupt, guide, tempt, transform, and eventually become part of someone’s life.

The player may become different kinds of entities through their choices.

Major influence identities:

1. **Manipulator**

2. Subtle control

3. Psychological nudges

4. Strategic pressure

5. Hidden influence

6. **Guardian**

7. Protection

8. Support

9. Stability

10. Healing and rescue

11. **Tempter**

12. Desire

- 13. Risk
- 14. Chaos
- 15. Escalation and temptation
- 16. **Creator**
- 17. Growth
- 18. Transformation
- 19. Identity shaping
- 20. Unlocking potential

The player does not choose one class at the beginning. The player becomes one through repeated action.

Core identity rule:

You are what you repeatedly influence.

6. Core Gameplay Loop

6.1 World Layer

The player observes the world from an isometric top-down view.

The player can:

- Watch characters move through the world
- Inspect character states
- Notice opportunities
- Influence multiple characters
- Trigger small events
- Push characters toward different choices
- Shape relationships between characters
- Build alignment through repeated influence styles

The world layer creates strategy, scale, and replayability.

6.2 Focus Layer

At any time, the player may focus attention on a specific character.

Focus mode allows:

- Deeper inspection
- Stronger influence
- Better emotional reading
- More precise interactions
- Character-specific events
- Connection growth
- Moment unlocks

The focus layer creates emotional attachment and depth.

6.3 Attention System

Attention is a limited resource.

The player cannot deeply influence everyone at the same time.

This creates strategic tension:

- Spread attention to affect the whole world
- Focus attention to deepen one relationship
- Neglect some characters to pursue others
- Shape broad events or personal outcomes

Attention is one of the most important systems in the game.

7. Relationship and Intimacy Design

The game may contain romantic, intimate, emotionally charged, seductive, or adult-adjacent moments, but these must be handled through a structured design system.

Important rules:

- All romance/intimacy-capable characters must be adults.
- Intimacy must never be framed as forced.
- The player may influence conditions, but must not override consent.
- Characters must have boundaries, preferences, resistance, and agency.
- Intimate moments are not the core game. They are high-value outcomes of deeper systemic play.
- The system should reward emotional context, trust, attraction, timing, and mutuality.
- Cheap “grind bar to unlock scene” design should be avoided.

Correct structure:

Influence creates conditions.
Conditions create context.
Context may unlock closeness.
Closeness requires character agency.

8. Core Systems

8.1 Alignment System

The player has hidden or semi-hidden alignment values.

Initial alignment model:

```
type PlayerAlignment = {  
  manipulator: number;  
  guardian: number;  
  tempter: number;  
  creator: number;  
};
```

Every influence action changes alignment.

Example:

```
{  
  id: "steady_presence",  
  alignment: {  
    guardian: 2  
  }  
}
```

Alignment affects:

- Available abilities
 - Event outcomes
 - Character reactions
 - Moment requirements
 - World reputation
 - Expansion-specific paths
 - Player identity
-

8.2 Influence Actions

Influence actions are the player's main way of interacting with the world.

Initial actions:

1. Nudge Thought

2. Small mental push

3. Low cost

4. Manipulator leaning

5. Amplify Emotion

6. Intensifies current emotional state

7. Riskier

8. Tempter leaning

9. Steady Presence

10. Calms, protects, stabilises

11. Guardian leaning

12. Spark Change

13. Encourages growth, curiosity, transformation

14. Creator leaning

Influence actions must be data-driven.

Example structure:

```
type InfluenceAction = {  
  id: string;  
  name: string;  
  description: string;  
  cost: number;  
  alignment: Partial<PlayerAlignment>;  
  validTargets: string[];
```

```
    effects: InfluenceEffect[];
};
```

8.3 Character State System

Every character has a combination of:

- Static traits
- Dynamic states
- Desires
- Fears
- Current location
- Current routine
- Current relationships
- Evolution paths
- Memory fragments

Example:

```
type Character = {
  id: string;
  name: string;
  age: number;
  role: string;
  traits: Record<string, number>;
  state: Record<string, number>;
  desires: string[];
  fears: string[];
  defaultLocation: string;
  evolutionPaths: string[];
};
```

8.4 Connection System

Connection is tracked per character.

Possible connection values:

```
type CharacterConnection = {
  trust: number;
  curiosity: number;
```



```
dependence: number;  
attraction: number;  
comfort: number;  
resistance: number;  
};
```

Connection should not be a single “love meter.” It must have multiple dimensions so that different playstyles create different outcomes.

8.5 Moment Event System

Moment events are high-value emotional, narrative, romantic, dramatic, or intimate events.

They are unlocked by conditions.

Example requirements:

- Character trust
- Character curiosity
- Player alignment
- Recent events
- Emotional state
- Time of day
- Location
- Relationship status
- Character boundaries
- Expansion world rules

Example:

```
type MomentEvent = {  
  id: string;  
  title: string;  
  requirements: MomentRequirements;  
  outcome: MomentOutcome;  
  tone: "emotional" | "romantic" | "dramatic" | "seductive" | "conflict" |  
  "revelation";  
};
```

Moment events should be earned, not spammed.

8.6 Character Evolution System

Characters change over time based on the player's influence.

Possible evolution paths:

- Breakthrough
- Burnout
- Obsession
- Independence
- Dependence
- Awakening
- Corruption
- Healing
- Ambition
- Isolation
- Confidence
- Collapse

The same character should be able to evolve differently depending on how the player interacts with them.

Core design rule:

The character becomes a reflection of the player's influence.

8.7 Relationship Between Characters

Later versions should include relationships between NPCs.

Examples:

- Friendship
- Rivalry
- Romance
- Professional tension
- Family pressure
- Jealousy
- Mentorship
- Dependency
- Competition

The player may influence not only individuals, but social dynamics.

This creates world-scale strategy.

8.8 LLM Character Mind Layer

The game may eventually use a very small local or hosted LLM to generate character flavour.

Important principle:

Systems decide what happens.
The LLM decides how it feels.

The LLM must not control core gameplay outcomes directly.

The LLM may be used for:

- Inner thoughts
- Dialogue flavour
- Moment descriptions
- Reactive journal entries
- Character memory summaries
- Unique emotional responses

The deterministic game system remains the authority for:

- Stats
- Outcomes
- Unlocks
- Progression
- Consent gates
- Character state
- Economy
- Save data

The LLM layer must be optional and replaceable.

If no LLM is available, the game must still work with templated dialogue.

9. First Starting Characters

The first MVP should include five characters.

9.1 Lena

Role: Corporate climber

Archetype: Overwhelmed achiever

Primary tension: ambition versus burnout
Strong reaction paths: Guardian, Manipulator

Traits:

```
{  
  ambition: 8,  
  anxiety: 7,  
  openness: 4  
}
```

State:

```
{  
  stress: 6,  
  confidence: 5,  
  trust: 2  
}
```

Desires:

- Success
- Recognition
- Stability

Fears:

- Failure
- Being replaced

Evolution paths:

- Burnout
- Breakthrough
- Dependence

9.2 Jay

Role: Unemployed creative / drifter
Archetype: Restless dreamer
Primary tension: freedom versus discipline
Strong reaction paths: Creator, Tempter

Traits:

```
{
  ambition: 3,
  curiosity: 8,
  discipline: 2
}
```

State:

```
{
  motivation: 3,
  mood: 5,
  attachment: 1
}
```

Desires:

- Freedom
- Expression

Fears:

- Commitment
- Routine

Evolution paths:

- Artist
- Lost
- Dependent

9.3 Maya

Role: Event organiser

Archetype: Social core

Primary tension: attention versus insecurity

Strong reaction paths: Manipulator, Tempter

Traits:

```
{
  charisma: 9,
  empathy: 7,
}
```

```
    insecurity: 5
  }
```

State:

```
{
  socialEnergy: 8,
  validationNeed: 6,
  trust: 3
}
```

Desires:

- Connection
- Attention

Fears:

- Rejection
- Being ignored

Evolution paths:

- Leader
- Obsessed
- Burnout

9.4 Ethan

Role: Engineer

Archetype: Stable one

Primary tension: control versus emotional openness

Strong reaction paths: Creator, Guardian

Traits:

```
{
  logic: 9,
  emotionalExpression: 3,
  loyalty: 8
}
```

State:

```
{
  routine: 9,
  curiosity: 2,
  trust: 4
}
```

Desires:

- Stability
- Control

Fears:

- Uncertainty
- Emotional chaos

Evolution paths:

- Opened
- Rigid
- Dependent

9.5 Zara

Role: Nightlife risk-taker

Archetype: The edge

Primary tension: intensity versus stability

Strong reaction paths: Tempter, Creator

Traits:

```
{
  impulsiveness: 9,
  confidence: 7,
  empathy: 3
}
```

State:

```
{
  thrill: 8,
  stability: 2,
}
```

```
attachment: 2  
}
```

Desires:

- Intensity
- Freedom

Fears:

- Boredom
- Control

Evolution paths:

- Self-destruction
- Awakening
- Obsession

10. Art Direction

10.1 Visual Style

The game should use a stylised isometric look.

The first version may use placeholder art, but the style direction should be clear.

Visual direction:

- Isometric city block
- Slightly exaggerated modern characters
- Clean silhouettes
- Readable emotional states
- Stylish colour-coded influence effects
- UI-heavy strategic clarity
- Lightly cinematic moment panels
- Characters should look attractive, distinct, and memorable without requiring explicit visuals

10.2 MVP Graphics

The MVP may use:

- Simple diamond isometric tiles

- Placeholder character sprites
- Coloured circles or simple avatars
- Basic buildings
- Simple walking loops
- UI panels for character info
- Floating status icons
- Influence effect particles

The agent must prioritise function over polish in early versions.

10.3 Future Graphics

Later art systems may include:

- Character portraits
 - Mood expressions
 - Unlockable visual states
 - Outfit variants
 - Expansion-specific skins
 - Moment event illustrations
 - Isometric animated environments
 - Weather and time-of-day effects
-

11. Technical Stack

The recommended technical stack is:

- React
- Vite
- TypeScript
- Phaser
- Zustand
- Zod
- Vitest
- Playwright
- Vercel for frontend deployment
- Supabase or custom backend for accounts, saves, progression, analytics, and content data

React handles:

- HUD
- Menus
- Character panels
- Settings

- Store screens
- Meta progression
- Dialogue/moment UI

Phaser handles:

- Isometric map
- Character movement
- World rendering
- Effects
- Selection
- Camera
- Game loop

TypeScript is mandatory.

The game must be modular and data-driven.

12. Repository Structure

Recommended structure:

```
/src
/app
  App.tsx
  routes.tsx

/game
  PhaserGame.ts
  scenes/
    BootScene.ts
    WorldScene.ts
  iso/
    iso-utils.ts
    depth-sort.ts
  entities/
    CharacterSprite.ts
  systems/
    movement-system.ts
    influence-system.ts
    connection-system.ts
    event-system.ts
    alignment-system.ts
    attention-system.ts
    moment-system.ts
```

```
    llm-flavour-system.ts
data/
  characters.ts
  influence-actions.ts
  events.ts
  moment-events.ts
  traits.ts
  worlds.ts
types/
  character.ts
  influence.ts
  alignment.ts
  world.ts
  moment.ts

/ui
  components/
    HUD.tsx
    CharacterPanel.tsx
    InfluenceMenu.tsx
    MomentPanel.tsx
  stores/
    useGameStore.ts

/backend
  api/
  db/
  services/

/docs
  GAME_AGENT_RDP.md
  PRD.md
  ARCHITECTURE.md
  CONTENT_GUIDE.md
  ROADMAP.md

/tasks
  task-001-project-setup.md
  task-002-isometric-grid.md
  task-003-character-system.md
  task-004-influence-actions.md
  task-005-moment-event.md
```

13. Backend Responsibilities

The backend is not required for the first offline prototype, but it should be planned from the beginning.

Backend responsibilities may include:

- User accounts
- Save games
- Player progression
- Purchased expansions
- Character unlocks
- World packs
- Content packs
- Analytics
- A/B testing
- Remote config
- LLM request handling
- Safety filtering for generated text
- Payment entitlement checks
- Cloud saves

For MVP, local storage is acceptable.

For production, use Supabase or a custom backend.

14. Deployment Responsibilities

The agent must be able to help deploy the game.

Primary deployment target:

- Vercel for frontend

Possible future targets:

- Netlify
- Static hosting
- Capacitor-wrapped mobile build
- Expo-based mobile companion
- Desktop wrapper if needed

Deployment expectations:

- Build must pass before deployment
- TypeScript errors must be fixed

- Tests should pass
 - Agent must report deployment status
 - Agent must provide deployed URL when available
 - Agent must not expose secrets
 - Environment variables must be documented clearly
-

15. Agent Role Definition

The agent is responsible for acting as the autonomous game designer, developer, technical planner, and build operator for **Above Us**.

The agent's job is to turn high-level direction from Klaasvaakie into working game systems.

The agent should be able to:

- Read the repository
- Understand the architecture
- Follow the rules in AGENTS.md
- Create new files
- Edit existing files
- Add game systems
- Add characters
- Add events
- Add UI components
- Add tests
- Run builds
- Run tests
- Prepare commits
- Prepare pull requests
- Assist with deployment
- Maintain documentation
- Suggest next tasks
- Protect project structure
- Avoid overbuilding too early

The agent must act as a production worker, not a brainstorming chatbot.

16. Agent Responsibilities

16.1 Game Design

The agent must:

- Maintain the core concept
 - Design influence actions
 - Design character systems
 - Design moment events
 - Design alignment paths
 - Design player progression
 - Design expansion-friendly content
 - Keep intimacy as a consequence of systems, not a cheap objective
 - Maintain consent, agency, and adult-character boundaries
 - Keep the game strategic and replayable
-

16.2 Coding

The agent must:

- Use TypeScript
 - Keep code modular
 - Avoid large unreviewable changes
 - Prefer small systems with clear interfaces
 - Keep rendering separate from logic
 - Keep UI separate from Phaser where possible
 - Add types before complex features
 - Use data files for characters and actions
 - Avoid hardcoded one-off logic
 - Run tests/builds when possible
-

16.3 Graphics and Visual Implementation

The agent must:

- Implement fake isometric projection for MVP
- Use placeholder graphics before polished graphics
- Keep sprites replaceable
- Keep visual state readable
- Use simple visual effects for influence actions
- Add character selection highlighting
- Use depth sorting for isometric visuals

- Keep art pipeline simple

The agent should not delay gameplay work because final art is unavailable.

16.4 UI/UX

The agent must:

- Build readable player HUD
 - Show selected character info
 - Show available influence actions
 - Show feedback when influence is applied
 - Show alignment progression when appropriate
 - Show moment events clearly
 - Keep the UI usable on desktop first
 - Plan for mobile later, but do not prioritise mobile before the core loop works
-

16.5 Narrative and Content

The agent must:

- Write character descriptions
 - Create event chains
 - Create moment event templates
 - Generate character dialogue where needed
 - Maintain tone consistency
 - Avoid explicit content unless future platform strategy allows it
 - Keep all intimacy-capable characters adult
 - Make content reactive to systems
-

16.6 LLM Layer

The agent must:

- Treat LLM output as flavour, not authority
 - Keep deterministic systems in control
 - Create prompt templates for character thoughts and dialogue
 - Cache generated text where needed
 - Ensure fallback text exists
 - Avoid allowing generated text to break game rules
 - Store character memory summaries safely
-

16.7 Testing

The agent must:

- Add tests for core systems
 - Test influence effects
 - Test alignment changes
 - Test moment requirements
 - Test character state changes
 - Test data validation
 - Run build checks before marking work complete
-

16.8 Documentation

The agent must keep the following updated:

- AGENTS.md
- README.md
- ARCHITECTURE.md
- ROADMAP.md
- CONTENT_GUIDE.md
- TASK files
- CHANGELOG.md if used

The agent must document important decisions.

17. Agent Operating Rules

The agent must follow these rules:

1. Never destroy existing architecture without documenting why.
2. Never make huge sweeping rewrites unless explicitly instructed.
3. Prefer small, testable modules.
4. Prefer data-driven content over hardcoded logic.
5. Keep MVP simple.
6. Do not add monetisation before the core loop works.
7. Do not add multiple worlds before the modern world works.
8. Do not make intimacy the only goal.
9. Do not simulate everything. Use illusion, state machines, and event systems.
10. Always explain changed files after completing a task.
11. Always state what was tested.
12. Always list remaining risks.
13. Never commit secrets.

14. Never assume unavailable services exist.
 15. Ask for approval before adding paid services or major dependencies.
-

18. MVP Scope

The first playable prototype must include:

- Vite + React + TypeScript project
- Phaser integration
- One fake isometric map
- Five characters
- Character movement loops
- Character selection
- Character info panel
- Four influence actions
- Basic player alignment tracking
- Basic connection tracking
- Basic attention/focus system
- One moment event
- Local save or in-memory state
- Simple visual feedback
- Basic build/test pipeline

The MVP does not need:

- Final art
 - Authentication
 - Payments
 - Expansions
 - Real LLM integration
 - Multiplayer
 - Mobile app build
 - Full economy
 - Explicit adult content
 - Complex AI simulation
-

19. First Build Tasks

Task 001: Project Setup

Create a Vite + React + TypeScript project with Phaser installed.

Add initial folder structure.

Add README and AGENTS.md.

Task 002: Phaser Scene

Create a Phaser world scene.

Render a basic fake isometric grid.

Add camera support.

Task 003: Character Data

Add the five starting characters:

- Lena
- Jay
- Maya
- Ethan
- Zara

Each character must have:

- id
 - name
 - role
 - traits
 - state
 - desires
 - fears
 - default location
 - evolution paths
-

Task 004: Character Rendering

Render five placeholder character sprites or markers on the isometric grid.

Characters must have simple movement loops.

Task 005: Selection and UI

Allow the player to click a character.

Show selected character data in React UI.

Task 006: Influence System

Add four influence actions:

- Nudge Thought
- Amplify Emotion
- Steady Presence
- Spark Change

Applying an action must change character state and player alignment.

Task 007: Connection System

Track connection values per character.

Start with:

- trust
 - curiosity
 - dependence
 - attraction
 - comfort
 - resistance
-

Task 008: First Moment Event

Create one event called `first_connection`.

It unlocks when requirements are met.

It should display a simple moment panel.

Task 009: Test and Build

Add basic tests for:

- influence actions
- alignment changes
- moment requirements

- character state updates

Ensure the app builds.

20. Monetisation Direction

The monetisation should be designed carefully.

Potential monetisation:

- Expansion worlds
- Premium character arcs
- Cosmetic variants
- Alternate evolution paths
- Faster progression
- Rare moment packs
- Optional supporter packs
- Cloud save / premium profile features
- Seasonal world events

Avoid:

- Selling intimacy directly
- Making the game pay-to-win
- Locking the whole emotional payoff behind payment
- Creating platform-risk content too early

Best monetisation frame:

Players pay to explore more versions of themselves across more worlds.

21. World Pack Architecture

Each world pack should include:

```
/worlds
  /modern
    characters.ts
    events.ts
    traits.ts
    moment-events.ts
    visuals.ts
    rules.ts
```

```
/medieval
  characters.ts
  events.ts
  traits.ts
  moment-events.ts
  visuals.ts
  rules.ts

/cyberpunk
  characters.ts
  events.ts
  traits.ts
  moment-events.ts
  visuals.ts
  rules.ts
```

The engine must be able to load a world pack and use the same core systems.

22. Content Safety and Platform Rules

The game may include sensuality, attraction, tension, flirtation, romantic situations, and emotionally intimate moments.

The game must avoid:

- Underage sexual content
- Coercive sexual framing
- Explicit non-consensual situations
- Sexual exploitation
- Forced intimacy
- Characters without agency
- Content that exists only to bypass platform rules

The player can influence conditions, but characters must retain agency.

This is both a design rule and a platform survival rule.

23. Success Criteria

The first successful prototype should let a tester say:

- “I understand that I am influencing people from above.”

- "I can select a character and affect them."
- "I can see the character change."
- "I can feel that my choices are shaping what kind of entity I am."
- "I want to see what happens if I keep influencing them."
- "I can imagine other worlds using the same system."

24. Agent Startup Instruction

When the agent begins work, it must read this document and then execute the project in phases.

Initial command:

```
You are the autonomous development and design agent for Above Us.
```

```
Read /docs/GAME_AGENT_RDP.md and follow it strictly.
```

```
Your first task is to create the MVP foundation:
```

- Vite + React + TypeScript
- Phaser integration
- fake isometric grid
- five placeholder characters
- selectable character system
- React character info panel
- basic influence action structure

```
Keep everything modular and data-driven.
```

```
Do not add final art.
```

```
Do not add monetisation.
```

```
Do not add backend yet.
```

```
After completing the task, report changed files, tests run, and next recommended task.
```

25. Required Agent Tooling

To operate properly, the agent needs access to:

1. A GitHub repository
2. Local or cloud code execution
3. File editing permission
4. Terminal command permission
5. Build/test command permission

6. Git branch and commit permission
7. Vercel deployment access, when deployment begins
8. Backend project access, when backend begins
9. Documentation files inside the repo
10. A task queue using markdown files or GitHub Issues

Recommended operational setup:

```
GitHub repo
+ Codex or Cursor Agent
+ AGENTS.md
+ /docs folder
+ /tasks folder
+ Vercel
+ Supabase or custom backend later
```

Important note:

The agent cannot automatically access ChatGPT internal skills unless those skills are exposed through an integration layer, copied into repo documentation, or manually used by Klaasvaakie/Sani during planning. For now, the safe approach is to place all critical instructions inside the repo as markdown documents.

26. Final North Star

Above Us is not a simple adult game.

It is not only a god game.

It is not only a dating sim.

It is a system where the player becomes an unseen force, shapes lives from above, and discovers their own nature through the consequences of influence.

The game's strongest line:

Shape them. Watch them change. Discover what you are.

(| \)
(| \)